

In [1]: *#Assignment No 1*

In [2]: *# Data manipulation*
import pandas **as** pd
import numpy **as** np

In [3]: *# Data Visualization*
import matplotlib.pyplot **as** plt

In [4]: *# Machine Learning Utilities*
from sklearn.model_selection **import** train_test_split
from sklearn.preprocessing **import** LabelEncoder
from sklearn.preprocessing **import** StandardScaler

In [5]: *# Evaluation Metrics*
from sklearn.metrics **import** (
 mean_absolute_error,
 mean_squared_error,
 r2_score
)

In [6]: *# TensorFlow / Keras Libraries*
from tensorflow.keras.models **import** Sequential
from tensorflow.keras.layers **import** Dense
from tensorflow.keras.callbacks **import** EarlyStopping

In [7]: *# Step 2 : Load Dataset*

Read the CSV file

 df = pd.read_csv("House Price Prediction Dataset.csv")

Display first five rows

 df.head()

Out[7]:

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location	Condition	Garage	
0	1	1360	5	4	3	1970	Downtown	Excellent	No	1
1	2	4272	5	4	3	1958	Downtown	Excellent	No	4
2	3	3592	2	2	3	1938	Downtown	Good	No	2
3	4	966	4	2	2	1902	Suburban	Fair	Yes	2
4	5	4926	1	4	2	1975	Downtown	Fair	Yes	6



In [8]: *# Step 3 : Dataset Exploration*

Display dataset information

 print("Shape of Dataset :", df.shape)

```
print("\nColumn Names\n")
print(df.columns)

print("\nDataset Information\n")
print(df.info())

print("\nStatistical Summary\n")
print(df.describe())

print("\nMissing Values\n")
print(df.isnull().sum())
```

Shape of Dataset : (2000, 10)

Column Names

```
Index(['Id', 'Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt',
      'Location', 'Condition', 'Garage', 'Price'],
      dtype='object')
```

Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 10 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Id          2000 non-null   int64
 1   Area        2000 non-null   int64
 2   Bedrooms    2000 non-null   int64
 3   Bathrooms   2000 non-null   int64
 4   Floors      2000 non-null   int64
 5   YearBuilt   2000 non-null   int64
 6   Location    2000 non-null   object
 7   Condition   2000 non-null   object
 8   Garage      2000 non-null   object
 9   Price       2000 non-null   int64
dtypes: int64(7), object(3)
memory usage: 156.4+ KB
None
```

Statistical Summary

	Id	Area	Bedrooms	Bathrooms	Floors \
count	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000
mean	1000.500000	2786.209500	3.003500	2.55250	1.993500
std	577.494589	1295.146799	1.424606	1.10899	0.809188
min	1.000000	501.000000	1.000000	1.00000	1.000000
25%	500.750000	1653.000000	2.000000	2.00000	1.000000
50%	1000.500000	2833.000000	3.000000	3.00000	2.000000
75%	1500.250000	3887.500000	4.000000	4.00000	3.000000
max	2000.000000	4999.000000	5.000000	4.00000	3.000000

	YearBuilt	Price
count	2000.000000	2000.000000
mean	1961.446000	537676.855000
std	35.926695	276428.845719
min	1900.000000	50005.000000
25%	1930.000000	300098.000000
50%	1961.000000	539254.000000
75%	1993.000000	780086.000000
max	2023.000000	999656.000000

Missing Values

Id	0
Area	0
Bedrooms	0
Bathrooms	0
Floors	0
YearBuilt	0
Location	0

```
Condition    0
Garage       0
Price        0
dtype: int64
```

```
In [9]: # Step 4 : Handle Missing Values

# Fill numerical missing values using median

num_cols = df.select_dtypes(include=['int64', 'float64']).columns

for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)

# Fill categorical missing values using mode

cat_cols = df.select_dtypes(include=['object']).columns

for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

print("Missing Values After Cleaning\n")

print(df.isnull().sum())
```

Missing Values After Cleaning

```
Id          0
Area        0
Bedrooms    0
Bathrooms   0
Floors       0
YearBuilt    0
Location     0
Condition    0
Garage       0
Price        0
dtype: int64
```

C:\Users\HP\AppData\Local\Temp\ipykernel_18468\3104745437.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].median(), inplace=True)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_18468\3104745437.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].median(), inplace=True)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_18468\3104745437.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].median(), inplace=True)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_18468\3104745437.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].median(), inplace=True)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_18468\3104745437.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.meth

`od({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.`

```
df[col].fillna(df[col].median(), inplace=True)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_18468\3104745437.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing '`df[col].method(value, inplace=True)`', try using '`df.method({col: value}, inplace=True)`' or '`df[col] = df[col].method(value)`' instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].median(), inplace=True)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_18468\3104745437.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing '`df[col].method(value, inplace=True)`', try using '`df.method({col: value}, inplace=True)`' or '`df[col] = df[col].method(value)`' instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].median(), inplace=True)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_18468\3104745437.py:15: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing '`df[col].method(value, inplace=True)`', try using '`df.method({col: value}, inplace=True)`' or '`df[col] = df[col].method(value)`' instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].mode()[0], inplace=True)
```

```
In [10]: # Step 5 : Encode Categorical Variables
encoder = LabelEncoder()

# Convert text columns into numeric values

for col in cat_cols:
    df[col] = encoder.fit_transform(df[col])

df.head()
```

Out[10]:

	Id	Area	Bedrooms	Bathrooms	Floors	YearBuilt	Location	Condition	Garage	
0	1	1360	5	4	3	1970	0	0	0	14
1	2	4272	5	4	3	1958	0	0	0	42
2	3	3592	2	2	3	1938	0	2	0	26
3	4	966	4	2	2	1902	2	1	1	24
4	5	4926	1	4	2	1975	0	1	1	63



```
In [11]: # Step 6 : Feature Selection

# Target Column

target = "Price"

# Independent Variables

X = df.drop(target, axis=1)

# Dependent Variable

y = df[target]

print("Feature Shape :", X.shape)

print("Target Shape :", y.shape)
```

Feature Shape : (2000, 9)
Target Shape : (2000,)

```
In [12]: # Step 7 : Split Dataset

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training Samples :", X_train.shape)

print("Testing Samples :", X_test.shape)
```

Training Samples : (1600, 9)
Testing Samples : (400, 9)

```
In [13]: # Step 8 : Standardization

# StandardScaler improves ANN performance

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

In [14]: *# Step 9 : Build Artificial Neural Network*

```
model = Sequential()

# Input Layer + Hidden Layer 1

model.add(Dense(
    units=64,
    activation='relu',
    input_shape=(X_train.shape[1],)
))

# Hidden Layer 2

model.add(Dense(
    units=32,
    activation='relu'
))

# Hidden Layer 3

model.add(Dense(
    units=16,
    activation='relu'
))

# Output Layer

model.add(Dense(
    units=1,
    activation='linear'
))
```

c:\Users\HP\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

In [15]: *# Step 10 : Compile Model*

```
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)

# Display Model Architecture

model.summary()
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	640
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17



Total params: 3,265 (12.75 KB)

Trainable params: 3,265 (12.75 KB)

Non-trainable params: 0 (0.00 B)

```
In [16]: # Step 11 : Train the ANN

# EarlyStopping prevents overfitting

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)

history = model.fit(
    X_train,
    y_train,
    validation_split=0.20,
    epochs=100,
    batch_size=16,
    callbacks=[early_stop],
    verbose=1
)
```

Epoch 1/100
80/80 ————— 3s 9ms/step - loss: 367025979392.0000 - mae: 538499.56
25 - val_loss: 349356359680.0000 - val_mae: 526881.3750
Epoch 2/100
80/80 ————— 0s 4ms/step - loss: 366927347712.0000 - mae: 538407.12
50 - val_loss: 349114859520.0000 - val_mae: 526652.1250
Epoch 3/100
80/80 ————— 1s 7ms/step - loss: 366282309632.0000 - mae: 537808.43
75 - val_loss: 347928068096.0000 - val_mae: 525524.3125
Epoch 4/100
80/80 ————— 0s 4ms/step - loss: 363970985984.0000 - mae: 535676.50
00 - val_loss: 344400330752.0000 - val_mae: 522157.7500
Epoch 5/100
80/80 ————— 0s 3ms/step - loss: 358285901824.0000 - mae: 530314.87
50 - val_loss: 336659677184.0000 - val_mae: 514691.0625
Epoch 6/100
80/80 ————— 0s 4ms/step - loss: 347060600832.0000 - mae: 519693.06
25 - val_loss: 322728689664.0000 - val_mae: 500974.0000
Epoch 7/100
80/80 ————— 0s 4ms/step - loss: 328421703680.0000 - mae: 501361.93
75 - val_loss: 300876201984.0000 - val_mae: 478650.3125
Epoch 8/100
80/80 ————— 0s 4ms/step - loss: 301078872064.0000 - mae: 473889.68
75 - val_loss: 270776139776.0000 - val_mae: 447020.5000
Epoch 9/100
80/80 ————— 0s 4ms/step - loss: 265335554048.0000 - mae: 437178.75
00 - val_loss: 233386541056.0000 - val_mae: 407172.5000
Epoch 10/100
80/80 ————— 0s 3ms/step - loss: 223660113920.0000 - mae: 393769.03
12 - val_loss: 191864242176.0000 - val_mae: 363337.3125
Epoch 11/100
80/80 ————— 0s 4ms/step - loss: 180355432448.0000 - mae: 348848.96
88 - val_loss: 152308187136.0000 - val_mae: 320852.6562
Epoch 12/100
80/80 ————— 0s 3ms/step - loss: 141620772864.0000 - mae: 308876.68
75 - val_loss: 118676832256.0000 - val_mae: 284812.1875
Epoch 13/100
80/80 ————— 0s 3ms/step - loss: 112333381632.0000 - mae: 277735.78
12 - val_loss: 95968206848.0000 - val_mae: 259982.6719
Epoch 14/100
80/80 ————— 0s 4ms/step - loss: 93948911616.0000 - mae: 258984.671
9 - val_loss: 83227262976.0000 - val_mae: 244241.3750
Epoch 15/100
80/80 ————— 0s 3ms/step - loss: 84401987584.0000 - mae: 248317.718
8 - val_loss: 77271826432.0000 - val_mae: 236372.8281
Epoch 16/100
80/80 ————— 0s 3ms/step - loss: 80187277312.0000 - mae: 243205.796
9 - val_loss: 75000430592.0000 - val_mae: 232825.2812
Epoch 17/100
80/80 ————— 0s 4ms/step - loss: 78489534464.0000 - mae: 241676.406
2 - val_loss: 74045964288.0000 - val_mae: 231364.4688
Epoch 18/100
80/80 ————— 0s 3ms/step - loss: 77779156992.0000 - mae: 240652.406
2 - val_loss: 73627475968.0000 - val_mae: 230770.1562
Epoch 19/100
80/80 ————— 0s 4ms/step - loss: 77458202624.0000 - mae: 240365.703
1 - val_loss: 73412624384.0000 - val_mae: 230480.7500
Epoch 20/100
80/80 ————— 0s 4ms/step - loss: 77183967232.0000 - mae: 239965.906
2 - val_loss: 73363767296.0000 - val_mae: 230471.4062

```

Epoch 21/100
80/80 ————— 1s 7ms/step - loss: 77050281984.0000 - mae: 239704.046
9 - val_loss: 7314091008.0000 - val_mae: 230380.4219
Epoch 22/100
80/80 ————— 0s 5ms/step - loss: 76932235264.0000 - mae: 239657.656
2 - val_loss: 73284976640.0000 - val_mae: 230418.7812
Epoch 23/100
80/80 ————— 0s 4ms/step - loss: 76807176192.0000 - mae: 239429.250
0 - val_loss: 73143853056.0000 - val_mae: 230332.0469
Epoch 24/100
80/80 ————— 1s 7ms/step - loss: 76767084544.0000 - mae: 239319.671
9 - val_loss: 73137872896.0000 - val_mae: 230354.9062
Epoch 25/100
80/80 ————— 0s 4ms/step - loss: 76674277376.0000 - mae: 239185.203
1 - val_loss: 73126862848.0000 - val_mae: 230278.7500
Epoch 26/100
80/80 ————— 0s 5ms/step - loss: 76637577216.0000 - mae: 239171.578
1 - val_loss: 73140469760.0000 - val_mae: 230347.9688
Epoch 27/100
80/80 ————— 0s 4ms/step - loss: 76562046976.0000 - mae: 239026.171
9 - val_loss: 73069068288.0000 - val_mae: 230390.0000
Epoch 28/100
80/80 ————— 0s 3ms/step - loss: 76535250944.0000 - mae: 238982.968
8 - val_loss: 73016819712.0000 - val_mae: 230351.6719
Epoch 29/100
80/80 ————— 1s 6ms/step - loss: 76476465152.0000 - mae: 238938.593
8 - val_loss: 73110634496.0000 - val_mae: 230346.0469
Epoch 30/100
80/80 ————— 0s 4ms/step - loss: 76441845760.0000 - mae: 238796.703
1 - val_loss: 73077719040.0000 - val_mae: 230360.9688
Epoch 31/100
80/80 ————— 0s 3ms/step - loss: 76409110528.0000 - mae: 238805.171
9 - val_loss: 73125003264.0000 - val_mae: 230409.5312
Epoch 32/100
80/80 ————— 0s 3ms/step - loss: 76341919744.0000 - mae: 238697.953
1 - val_loss: 73038905344.0000 - val_mae: 230288.0000
Epoch 33/100
80/80 ————— 0s 3ms/step - loss: 76330737664.0000 - mae: 238672.953
1 - val_loss: 73045524480.0000 - val_mae: 230335.2031
Epoch 34/100
80/80 ————— 0s 3ms/step - loss: 76220874752.0000 - mae: 238485.031
2 - val_loss: 73046892544.0000 - val_mae: 230313.6250
Epoch 35/100
80/80 ————— 0s 3ms/step - loss: 76230377472.0000 - mae: 238491.468
8 - val_loss: 73090662400.0000 - val_mae: 230433.8438
Epoch 36/100
80/80 ————— 0s 3ms/step - loss: 76155576320.0000 - mae: 238390.718
8 - val_loss: 73118359552.0000 - val_mae: 230474.0312
Epoch 37/100
80/80 ————— 0s 3ms/step - loss: 76116377600.0000 - mae: 238334.421
9 - val_loss: 73110421504.0000 - val_mae: 230425.4062
Epoch 38/100
80/80 ————— 0s 3ms/step - loss: 76086386688.0000 - mae: 238272.500
0 - val_loss: 73206448128.0000 - val_mae: 230561.8281

```

```

In [17]: # Step 12 : Evaluate Model

loss, mae = model.evaluate(X_test, y_test)

print("Test Loss :", loss)

```

```
print("Test MAE :", mae)
```

13/13 ————— 0s 4ms/step - loss: 82096668672.0000 - mae: 247680.4375
Test Loss : 82096668672.0
Test MAE : 247680.4375

In [18]: *# Step 13 : Predict House Prices*

```
y_pred = model.predict(X_test)
```

13/13 ————— 0s 12ms/step

In [19]: *# Step 14 : Performance Evaluation*

```
MAE = mean_absolute_error(y_test, y_pred)
```

```
MSE = mean_squared_error(y_test, y_pred)
```

```
RMSE = np.sqrt(MSE)
```

```
R2 = r2_score(y_test, y_pred)
```

```
print("Mean Absolute Error :", MAE)
```

```
print("Mean Squared Error :", MSE)
```

```
print("Root Mean Squared Error :", RMSE)
```

```
print("R2 Score :", R2)
```

Mean Absolute Error : 247680.4375
Mean Squared Error : 82096660480.0
Root Mean Squared Error : 286525.14807604585
R² Score : -0.05524301528930664

In [20]: *# Step 15 : Actual vs Predicted*

```
results = pd.DataFrame({  
    "Actual Price": y_test.values,  
    "Predicted Price": y_pred.flatten()  
})
```

```
results.head(20)
```

Out[20]:

	Actual Price	Predicted Price
0	514764	511845.25000
1	694256	526136.25000
2	66375	554608.81250
3	650243	510968.18750
4	223285	521748.68750
5	468127	552497.00000
6	513002	545636.31250
7	911525	488044.96875
8	723265	585079.25000
9	339416	531089.31250
10	913498	529960.68750
11	657493	548606.00000
12	259727	434911.40625
13	345187	468920.81250
14	688751	483416.68750
15	777513	566555.25000
16	506388	553832.43750
17	589618	570495.37500
18	984581	470849.50000
19	683801	494823.06250

```
In [21]: # Step 16 : Training Curve

plt.figure(figsize=(8,5))

plt.plot(history.history['loss'], label='Training Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

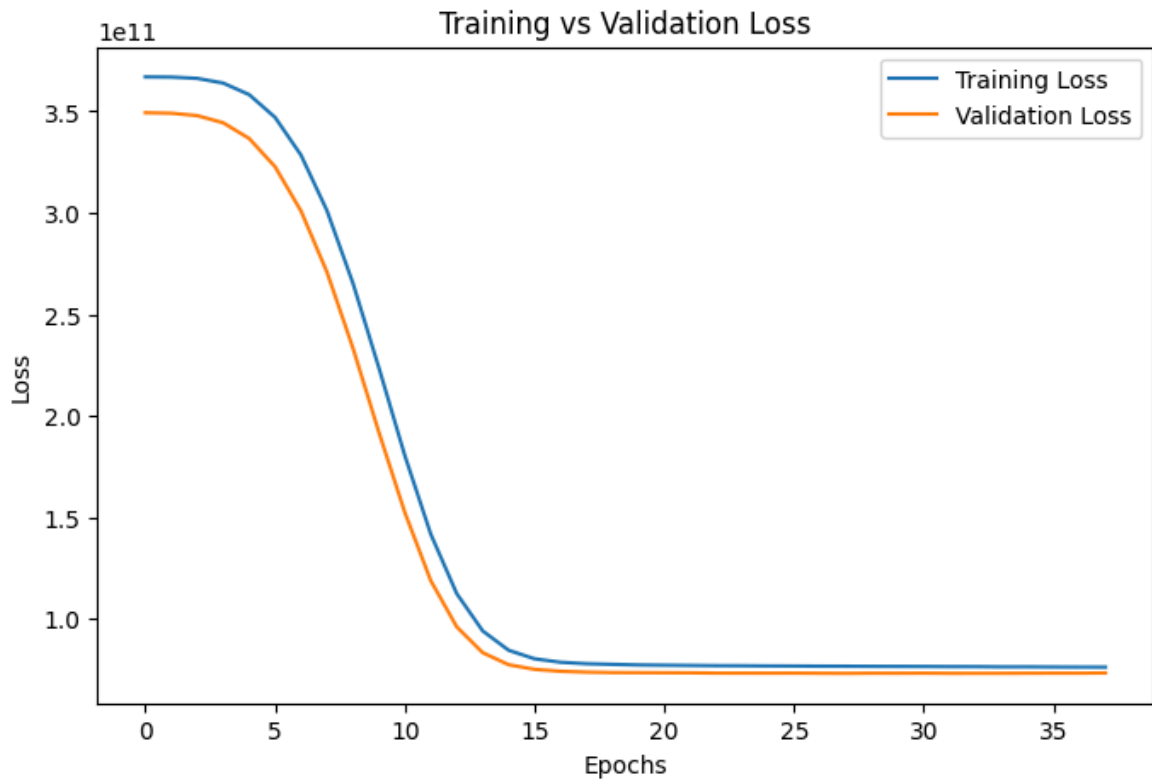
plt.xlabel("Epochs")

plt.ylabel("Loss")

plt.title("Training vs Validation Loss")

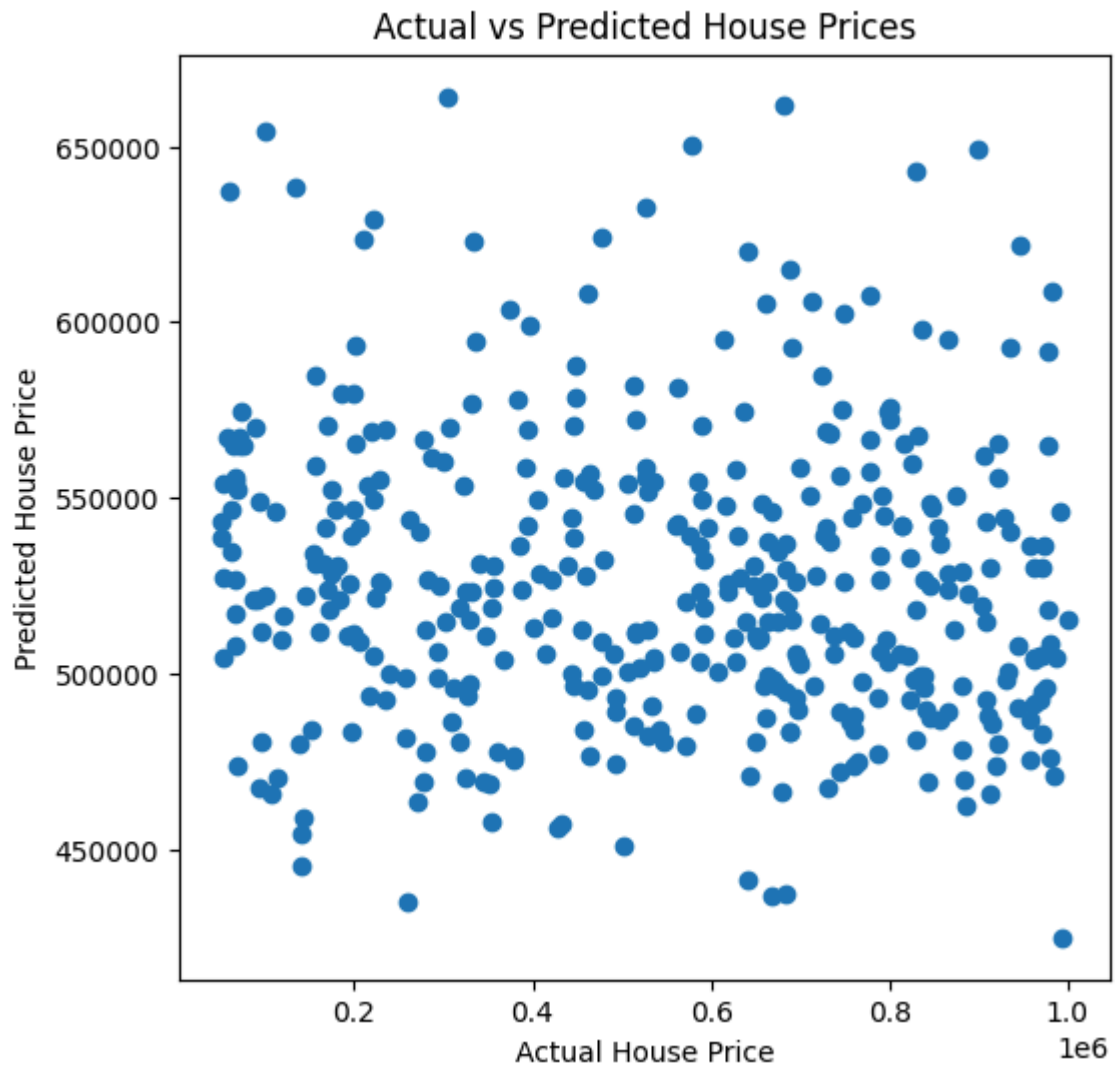
plt.legend()

plt.show()
```



In [22]: *# Step 17 : Scatter Plot*

```
plt.figure(figsize=(6,6))  
plt.scatter(y_test, y_pred)  
plt.xlabel("Actual House Price")  
plt.ylabel("Predicted House Price")  
plt.title("Actual vs Predicted House Prices")  
plt.show()
```



In []:

```
In [ ]: #Assignment No 01_Part B
```

```
In [ ]: #Step 1
        #import libraries
        import pandas as pd
        import numpy as np
```

```
In [2]: # Data Visualization
        import matplotlib.pyplot as plt
        import seaborn as sns
```

```
In [4]: # Machine Learning Utilities
        from sklearn.model_selection import train_test_split
        from sklearn.preprocessing import LabelEncoder
        from sklearn.preprocessing import StandardScaler
```

```
In [3]: # Evaluation Metrics
        from sklearn.metrics import (
            accuracy_score,
            precision_score,
            recall_score,
            f1_score,
            confusion_matrix,
            classification_report
        )
```

```
In [5]: # TensorFlow / Keras Libraries
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense, Input
        from tensorflow.keras.callbacks import EarlyStopping
```

```
In [9]: # Load Dataset
        # Read the CSV file
        df=pd.read_csv("D:\Advanced DS\Breast_Cancer Dataset.csv")

        # Display first five rows
        df.head()
```

```
<>:3: SyntaxWarning: invalid escape sequence '\A'
<>:3: SyntaxWarning: invalid escape sequence '\A'
C:\Users\HP\AppData\Local\Temp\ipykernel_11836\2452487104.py:3: SyntaxWarning: in
valid escape sequence '\A'
    df=pd.read_csv("D:\Advanced DS\Breast_Cancer Dataset.csv")
```


Out[9]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoo
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 33 columns



In [10]:

```
# Display dataset information

print("Shape of Dataset :", df.shape)

print("\nColumn Names\n")
print(df.columns)

print("\nDataset Information\n")
print(df.info())

print("\nStatistical Summary\n")
print(df.describe())

print("\nMissing Values\n")
print(df.isnull().sum())
```

Shape of Dataset : (569, 33)

Column Names

```
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst', 'Unnamed: 32'],
      dtype='object')
```

Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 569 entries, 0 to 568

Data columns (total 33 columns):

#	Column	Non-Null Count	Dtype
0	id	569 non-null	int64
1	diagnosis	569 non-null	object
2	radius_mean	569 non-null	float64
3	texture_mean	569 non-null	float64
4	perimeter_mean	569 non-null	float64
5	area_mean	569 non-null	float64
6	smoothness_mean	569 non-null	float64
7	compactness_mean	569 non-null	float64
8	concavity_mean	569 non-null	float64
9	concave points_mean	569 non-null	float64
10	symmetry_mean	569 non-null	float64
11	fractal_dimension_mean	569 non-null	float64
12	radius_se	569 non-null	float64
13	texture_se	569 non-null	float64
14	perimeter_se	569 non-null	float64
15	area_se	569 non-null	float64
16	smoothness_se	569 non-null	float64
17	compactness_se	569 non-null	float64
18	concavity_se	569 non-null	float64
19	concave points_se	569 non-null	float64
20	symmetry_se	569 non-null	float64
21	fractal_dimension_se	569 non-null	float64
22	radius_worst	569 non-null	float64
23	texture_worst	569 non-null	float64
24	perimeter_worst	569 non-null	float64
25	area_worst	569 non-null	float64
26	smoothness_worst	569 non-null	float64
27	compactness_worst	569 non-null	float64
28	concavity_worst	569 non-null	float64
29	concave points_worst	569 non-null	float64
30	symmetry_worst	569 non-null	float64
31	fractal_dimension_worst	569 non-null	float64
32	Unnamed: 32	0 non-null	float64

dtypes: float64(31), int64(1), object(1)

memory usage: 146.8+ KB

None

Statistical Summary

	id	radius_mean	texture_mean	perimeter_mean	area_mean	\
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
count	569.000000	569.000000	569.000000	569.000000	
mean	0.096360	0.104341	0.088799	0.048919	
std	0.014064	0.052813	0.079720	0.038803	
min	0.052630	0.019380	0.000000	0.000000	
25%	0.086370	0.064920	0.029560	0.020310	
50%	0.095870	0.092630	0.061540	0.033500	
75%	0.105300	0.130400	0.130700	0.074000	
max	0.163400	0.345400	0.426800	0.201200	

	symmetry_mean	...	texture_worst	perimeter_worst	area_worst	\
count	569.000000	...	569.000000	569.000000	569.000000	
mean	0.181162	...	25.677223	107.261213	880.583128	
std	0.027414	...	6.146258	33.602542	569.356993	
min	0.106000	...	12.020000	50.410000	185.200000	
25%	0.161900	...	21.080000	84.110000	515.300000	
50%	0.179200	...	25.410000	97.660000	686.500000	
75%	0.195700	...	29.720000	125.400000	1084.000000	
max	0.304000	...	49.540000	251.200000	4254.000000	

	smoothness_worst	compactness_worst	concavity_worst	\
count	569.000000	569.000000	569.000000	
mean	0.132369	0.254265	0.272188	
std	0.022832	0.157336	0.208624	
min	0.071170	0.027290	0.000000	
25%	0.116600	0.147200	0.114500	
50%	0.131300	0.211900	0.226700	
75%	0.146000	0.339100	0.382900	
max	0.222600	1.058000	1.252000	

	concave points_worst	symmetry_worst	fractal_dimension_worst	\
count	569.000000	569.000000	569.000000	
mean	0.114606	0.290076	0.083946	
std	0.065732	0.061867	0.018061	
min	0.000000	0.156500	0.055040	
25%	0.064930	0.250400	0.071460	
50%	0.099930	0.282200	0.080040	
75%	0.161400	0.317900	0.092080	
max	0.291000	0.663800	0.207500	

	Unnamed: 32
count	0.0
mean	NaN
std	NaN
min	NaN
25%	NaN
50%	NaN
75%	NaN
max	NaN

[8 rows x 32 columns]

Missing Values

```
id          0
diagnosis   0
radius_mean 0
texture_mean 0
perimeter_mean 0
area_mean   0
smoothness_mean 0
compactness_mean 0
concavity_mean 0
concave points_mean 0
symmetry_mean 0
fractal_dimension_mean 0
radius_se    0
texture_se    0
perimeter_se 0
area_se       0
smoothness_se 0
compactness_se 0
concavity_se  0
concave points_se 0
symmetry_se   0
fractal_dimension_se 0
radius_worst 0
texture_worst 0
perimeter_worst 0
area_worst    0
smoothness_worst 0
compactness_worst 0
concavity_worst 0
concave points_worst 0
symmetry_worst 0
fractal_dimension_worst 0
Unnamed: 32    569
dtype: int64
```

```
In [12]: # Check Target Column
# Display unique values of target column

print("Unique Diagnosis Values:")
print(df["diagnosis"].unique())

print("\nDiagnosis Value Counts:")
print(df["diagnosis"].value_counts())
```

Unique Diagnosis Values:
['M' 'B']

Diagnosis Value Counts:
diagnosis
B 357
M 212
Name: count, dtype: int64

```
In [13]: # Handle Missing Values
# Identify numerical and categorical columns
```

```

num_cols = df.select_dtypes(include=['int64', 'float64']).columns
cat_cols = df.select_dtypes(include=['object']).columns

# Fill numerical missing values using median
for col in num_cols:
    df[col] = df[col].fillna(df[col].median())

# Fill categorical missing values using mode
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

print("Missing Values After Cleaning\n")
print(df.isnull().sum())

```

Missing Values After Cleaning

```

id                                0
diagnosis                        0
radius_mean                      0
texture_mean                     0
perimeter_mean                   0
area_mean                        0
smoothness_mean                  0
compactness_mean                 0
concavity_mean                   0
concave points_mean              0
symmetry_mean                    0
fractal_dimension_mean           0
radius_se                        0
texture_se                       0
perimeter_se                     0
area_se                          0
smoothness_se                   0
compactness_se                   0
concavity_se                     0
concave points_se                0
symmetry_se                     0
fractal_dimension_se             0
radius_worst                     0
texture_worst                    0
perimeter_worst                  0
area_worst                       0
smoothness_worst                 0
compactness_worst                0
concavity_worst                  0
concave points_worst             0
symmetry_worst                   0
fractal_dimension_worst          0
Unnamed: 32                      569
dtype: int64

```

```

c:\Users\HP\AppData\Local\Programs\Python\Python313\Lib\site-packages\numpy\lib\
nanfunctions_impl.py:1214: RuntimeWarning: Mean of empty slice
  return np.nanmean(a, axis, out=out, keepdims=keepdims)

```

```

In [14]: # Encode Target Variable
encoder = LabelEncoder()

df["diagnosis"] = encoder.fit_transform(df["diagnosis"])

```

```
# Display encoded values
df.head()
```

Out[14]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoo
0	842302	1	17.99	10.38	122.80	1001.0	
1	842517	1	20.57	17.77	132.90	1326.0	
2	84300903	1	19.69	21.25	130.00	1203.0	
3	84348301	1	11.42	20.38	77.58	386.1	
4	84358402	1	20.29	14.34	135.10	1297.0	

5 rows × 33 columns

In [15]: *# Remove unnecessary columns if they exist*

```
columns_to_drop = []

if "id" in df.columns:
    columns_to_drop.append("id")

if "Unnamed: 32" in df.columns:
    columns_to_drop.append("Unnamed: 32")

df.drop(columns=columns_to_drop, inplace=True)

print("Remaining Columns:")
print(df.columns)
```

Remaining Columns:

```
Index(['diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst'],
      dtype='object')
```

In [16]: *# Target Column*
target = "diagnosis"

```
# Independent Variables
X = df.drop(target, axis=1)

# Dependent Variable
y = df[target]

print("Feature Shape :", X.shape)
print("Target Shape :", y.shape)
```

Feature Shape : (569, 30)

Target Shape : (569,)

```
In [17]: # Split dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Samples :", X_train.shape)
print("Testing Samples :", X_test.shape)
```

Training Samples : (455, 30)
Testing Samples : (114, 30)

```
In [18]: # Standardize the features

scaler = StandardScaler()

# Fit the scaler only on training data
X_train = scaler.fit_transform(X_train)

# Transform the test data using the same scaler
X_test = scaler.transform(X_test)

print("Feature Standardization Completed")
print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

Feature Standardization Completed
Training Data Shape: (455, 30)
Testing Data Shape: (114, 30)

```
In [19]: # Build Artificial Neural Network for Classification

model = Sequential()

# Input Layer
model.add(Input(shape=(X_train.shape[1],)))

# Hidden Layer 1
model.add(Dense(
    units=64,
    activation='relu'
))

# Hidden Layer 2
model.add(Dense(
    units=32,
    activation='relu'
))

# Hidden Layer 3
model.add(Dense(
    units=16,
    activation='relu'
))

# Output Layer
```

```
# Sigmoid is used for binary classification
model.add(Dense(
    units=1,
    activation='sigmoid'
))

print("ANN Classification Model Created")
```

ANN Classification Model Created

```
In [20]: # Sigmoid Function Visualization

import numpy as np
import matplotlib.pyplot as plt

# Define Sigmoid Function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Generate input values
x = np.linspace(-10, 10, 200)

# Calculate sigmoid output
y = sigmoid(x)

# Create the graph
plt.figure(figsize=(8, 5))

plt.plot(
    x,
    y,
    linewidth=2,
    label='Sigmoid Function'
)

# Decision threshold
plt.axhline(
    y=0.5,
    linestyle='--',
    label='Decision Threshold = 0.5'
)

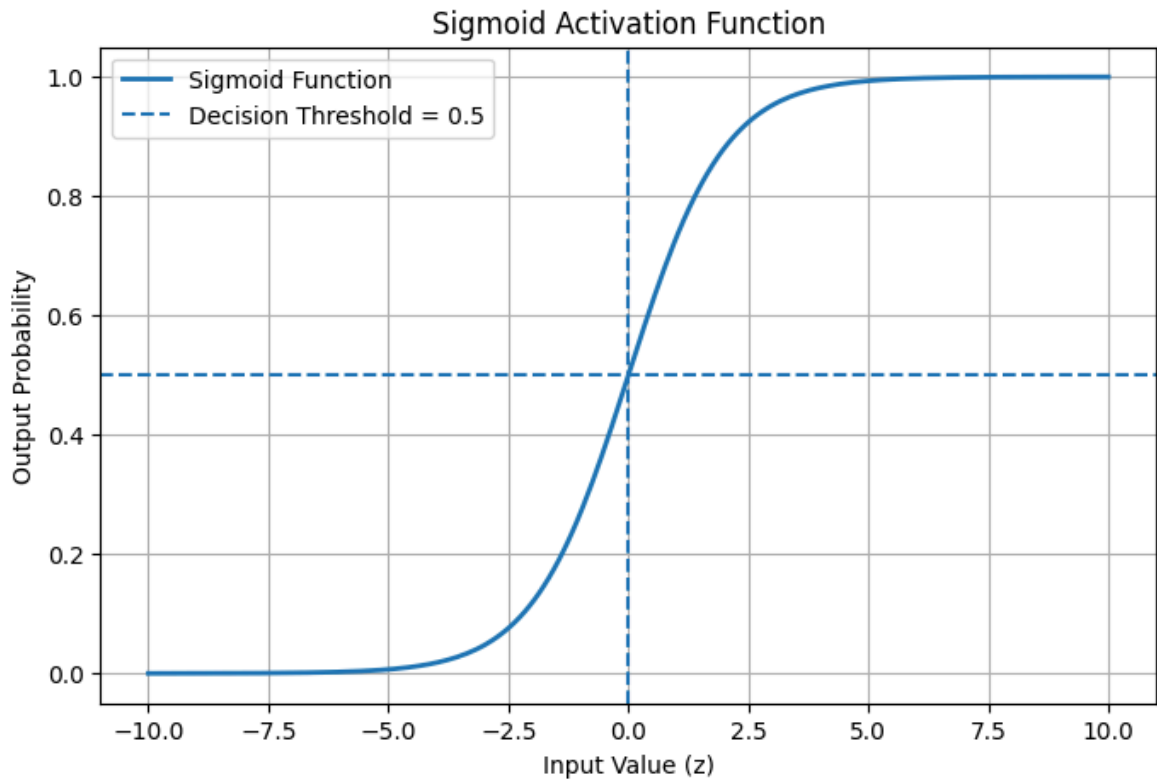
# Center point
plt.axvline(
    x=0,
    linestyle='--'
)

# Labels
plt.xlabel("Input Value (z)")
plt.ylabel("Output Probability")

plt.title("Sigmoid Activation Function")

plt.grid(True)
plt.legend()

plt.show()
```

In [21]: *# Display ANN Model Architecture*

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	1,984
dense_1 (Dense)	(None, 32)	2,080
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17



Total params: 4,609 (18.00 KB)

Trainable params: 4,609 (18.00 KB)

Non-trainable params: 0 (0.00 B)

In [22]: *# Compile the ANN Model*

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

print("Model Compilation Completed")
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Model Compilation Completed

In [23]: *# Early Stopping to reduce overfitting*

```
early_stop = EarlyStopping(  
    monitor='val_loss',  
    patience=10,  
    restore_best_weights=True  
)
```

Train the ANN Model

```
history = model.fit(  
    X_train,  
    y_train,  
    validation_split=0.20,  
    epochs=100,  
    batch_size=16,  
    callbacks=[early_stop],  
    verbose=1  
)
```

Epoch 1/100
23/23  4s 31ms/step - accuracy: 0.6923 - loss: 0.5551 - val_accuracy: 0.8681 - val_loss: 0.3650

Epoch 2/100
23/23  0s 9ms/step - accuracy: 0.9286 - loss: 0.3042 - val_accuracy: 0.9231 - val_loss: 0.2429

Epoch 3/100
23/23  0s 9ms/step - accuracy: 0.9670 - loss: 0.1833 - val_accuracy: 0.9560 - val_loss: 0.1714

Epoch 4/100
23/23  0s 8ms/step - accuracy: 0.9835 - loss: 0.1131 - val_accuracy: 0.9670 - val_loss: 0.1322

Epoch 5/100
23/23  0s 15ms/step - accuracy: 0.9835 - loss: 0.0806 - val_accuracy: 0.9560 - val_loss: 0.1171

Epoch 6/100
23/23  0s 9ms/step - accuracy: 0.9890 - loss: 0.0641 - val_accuracy: 0.9560 - val_loss: 0.1106

Epoch 7/100
23/23  0s 8ms/step - accuracy: 0.9863 - loss: 0.0550 - val_accuracy: 0.9560 - val_loss: 0.1071

Epoch 8/100
23/23  0s 12ms/step - accuracy: 0.9945 - loss: 0.0481 - val_accuracy: 0.9560 - val_loss: 0.1062

Epoch 9/100
23/23  0s 11ms/step - accuracy: 0.9945 - loss: 0.0408 - val_accuracy: 0.9670 - val_loss: 0.1057

Epoch 10/100
23/23  0s 13ms/step - accuracy: 0.9945 - loss: 0.0362 - val_accuracy: 0.9670 - val_loss: 0.1032

Epoch 11/100
23/23  0s 8ms/step - accuracy: 0.9945 - loss: 0.0322 - val_accuracy: 0.9451 - val_loss: 0.1077

Epoch 12/100
23/23  0s 8ms/step - accuracy: 0.9945 - loss: 0.0285 - val_accuracy: 0.9560 - val_loss: 0.1035

Epoch 13/100
23/23  0s 16ms/step - accuracy: 0.9945 - loss: 0.0259 - val_accuracy: 0.9451 - val_loss: 0.1095

Epoch 14/100
23/23  0s 11ms/step - accuracy: 0.9945 - loss: 0.0215 - val_accuracy: 0.9560 - val_loss: 0.1101

Epoch 15/100
23/23  0s 7ms/step - accuracy: 0.9945 - loss: 0.0177 - val_accuracy: 0.9560 - val_loss: 0.1161

Epoch 16/100
23/23  0s 8ms/step - accuracy: 0.9945 - loss: 0.0158 - val_accuracy: 0.9560 - val_loss: 0.1164

Epoch 17/100
23/23  0s 7ms/step - accuracy: 1.0000 - loss: 0.0130 - val_accuracy: 0.9341 - val_loss: 0.1234

Epoch 18/100
23/23  0s 9ms/step - accuracy: 0.9973 - loss: 0.0115 - val_accuracy: 0.9560 - val_loss: 0.1169

Epoch 19/100
23/23  0s 7ms/step - accuracy: 1.0000 - loss: 0.0098 - val_accuracy: 0.9451 - val_loss: 0.1206

Epoch 20/100
23/23  0s 9ms/step - accuracy: 1.0000 - loss: 0.0082 - val_accuracy: 0.9560 - val_loss: 0.1232

In [24]: *# Evaluate the model on test data*

```
loss, accuracy = model.evaluate(  
    X_test,  
    y_test,  
    verbose=0  
)  
  
print("Test Loss      :", loss)  
print("Test Accuracy :", accuracy)
```

Test Loss : 0.07095950841903687

Test Accuracy : 0.9824561476707458

In [25]: *# Predict probability values*

```
y_pred_probability = model.predict(X_test)  
  
# Convert probability into class labels  
  
y_pred = (  
    y_pred_probability >= 0.5  
)  
.astype(int).flatten()  
  
print("First 10 Predicted Probabilities:")  
print(y_pred_probability[:10])  
  
print("\nFirst 10 Predicted Classes:")  
print(y_pred[:10])
```

4/4 ————— 0s 32ms/step

First 10 Predicted Probabilities:

```
[[5.1447900e-05]  
 [9.9998719e-01]  
 [3.1454813e-02]  
 [5.4689211e-01]  
 [1.9526756e-01]  
 [3.1769171e-04]  
 [9.1769910e-01]  
 [4.2248730e-04]  
 [8.9767476e-05]  
 [4.2861723e-03]]
```

First 10 Predicted Classes:

```
[0 1 0 1 0 0 1 0 0 0]
```

In [26]: *# Calculate Classification Metrics*

```
accuracy = accuracy_score(  
    y_test,  
    y_pred  
)  
  
precision = precision_score(  
    y_test,  
    y_pred  
)  
  
recall = recall_score(  
    y_test,
```

```

    y_pred
)

f1 = f1_score(
    y_test,
    y_pred
)

print("Accuracy :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall    :", round(recall, 4))
print("F1 Score  :", round(f1, 4))

```

```

Accuracy : 0.9825
Precision : 1.0
Recall    : 0.9524
F1 Score  : 0.9756

```

In [27]: *# Display Classification Report*

```

print(
    classification_report(
        y_test,
        y_pred,
        target_names=encoder.classes_
    )
)

```

	precision	recall	f1-score	support
B	0.97	1.00	0.99	72
M	1.00	0.95	0.98	42
accuracy			0.98	114
macro avg	0.99	0.98	0.98	114
weighted avg	0.98	0.98	0.98	114

In [28]: *# Calculate Confusion Matrix*

```

cm = confusion_matrix(
    y_test,
    y_pred
)

print("Confusion Matrix:")
print(cm)

```

```

Confusion Matrix:
[[72  0]
 [ 2 40]]

```

In [29]: *# Visualize Confusion Matrix*

```

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',

```

```

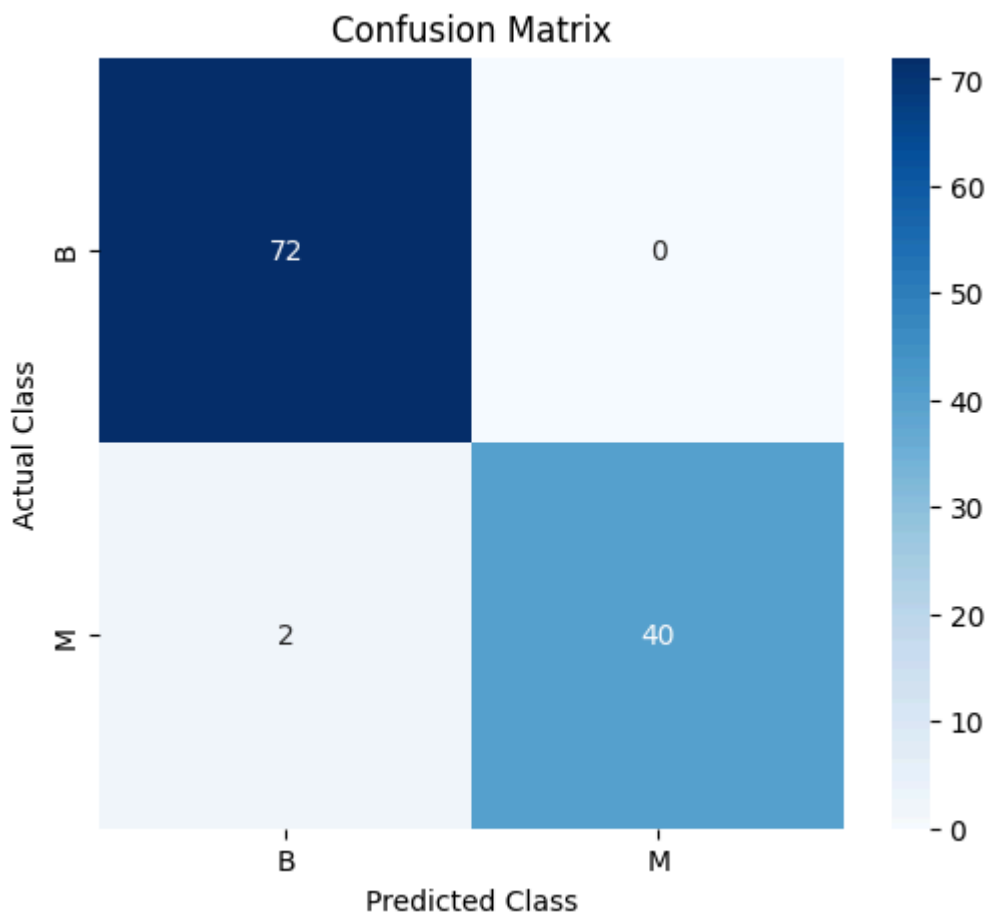
    xticklabels=encoder.classes_,
    yticklabels=encoder.classes_
)

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.title("Confusion Matrix")

plt.show()

```



In [30]: *# Plot Training and Validation Accuracy*

```

plt.figure(figsize=(8, 5))

plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)

plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)

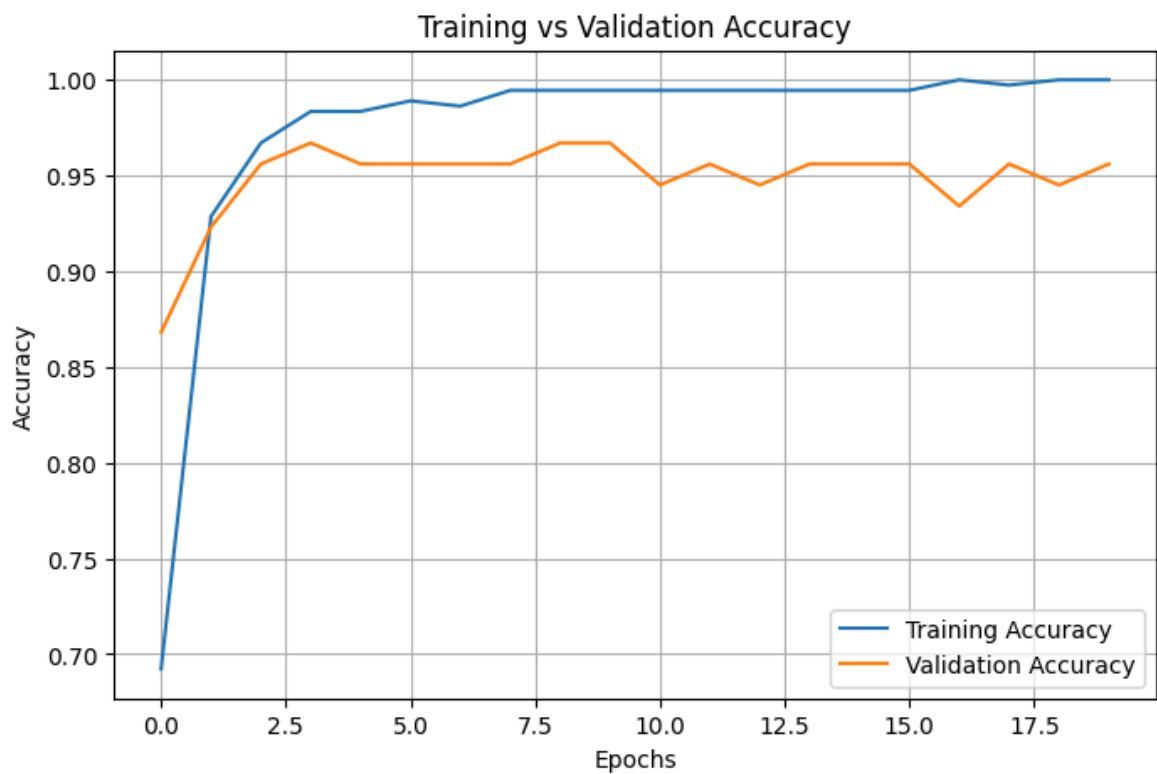
plt.xlabel("Epochs")
plt.ylabel("Accuracy")

plt.title("Training vs Validation Accuracy")

plt.legend()

```

```
plt.grid(True)  
plt.show()
```



In [31]: *# Final Model Performance Summary*

```
performance = pd.DataFrame({  
    "Metric": [  
        "Accuracy",  
        "Precision",  
        "Recall",  
        "F1 Score"  
    ],  
    "Score": [  
        accuracy,  
        precision,  
        recall,  
        f1  
    ]  
})  
  
performance
```

Out[31]:

	Metric	Score
0	Accuracy	0.982456
1	Precision	1.000000
2	Recall	0.952381
3	F1 Score	0.975610

In []:

